

Inference methods

Decoding, the memory wall, and serving on a cluster

Carlos Cotrini

ETH Zurich

7 October 2026

1

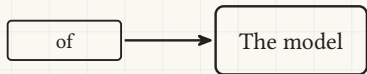
Section 1

Decoding as an algorithm

One token at a time

of

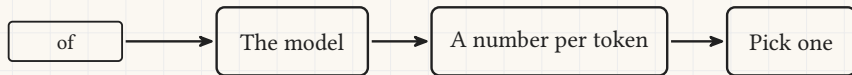
One token at a time



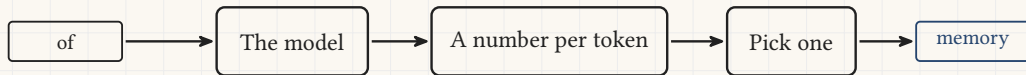
One token at a time



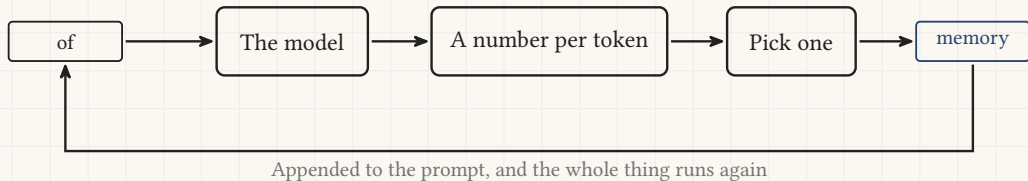
One token at a time



One token at a time



One token at a time



Always taking the most likely token makes the text worse

Human text

Always the top token

Always taking the most likely token makes the text worse

	Human text	Always the top token
Repeated phrases	0.28%	73.66%

Always taking the most likely token makes the text worse

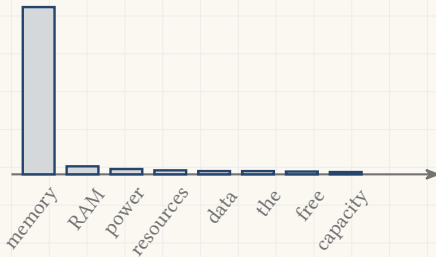
	Human text	Always the top token
Repeated phrases	0.28%	73.66%
Perplexity of what it wrote	12.38	1.50

Always taking the most likely token makes the text worse

	Human text	Always the top token
Repeated phrases	0.28%	73.66%
Perplexity of what it wrote	12.38	1.50
Past a beam of 64, it stops generating at once		
GPT-2 Large, 5000 passages		

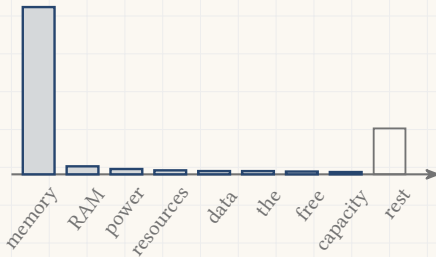
The same two knobs meet two very different distributions

"The GPU ran out of"



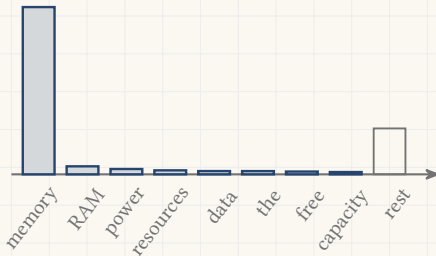
The same two knobs meet two very different distributions

"The GPU ran out of"



The same two knobs meet two very different distributions

"The GPU ran out of"

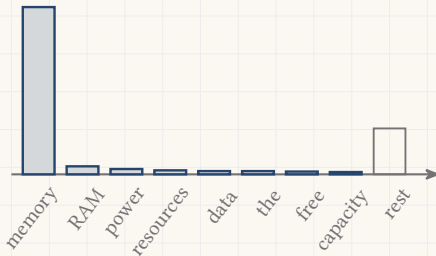


"To run a job on the cluster you first have to"

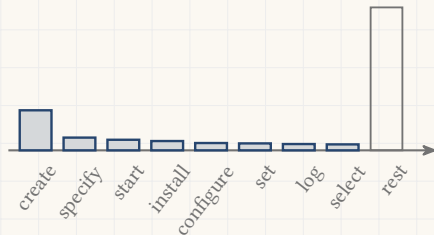


The same two knobs meet two very different distributions

"The GPU ran out of"

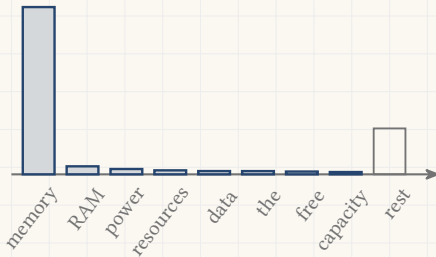


"To run a job on the cluster you first have to"



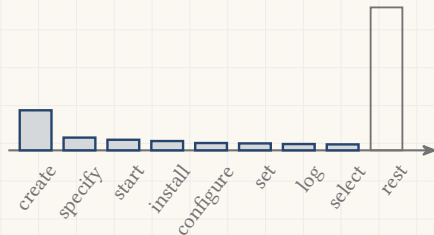
The same two knobs meet two very different distributions

"The GPU ran out of"



Top token 0.69

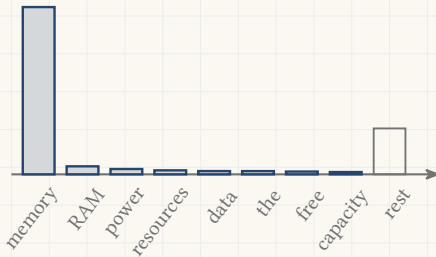
"To run a job on the cluster you first have to"



0.17

The same two knobs meet two very different distributions

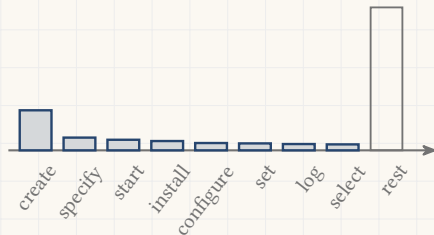
"The GPU ran out of"



Top token 0.69

Top-50 covers 91%

"To run a job on the cluster you first have to"

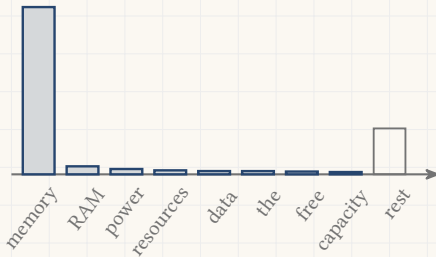


0.17

78%

The same two knobs meet two very different distributions

"The GPU ran out of"

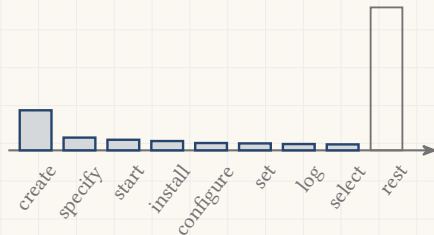


Top token 0.69

Top-50 covers 91%

Top-p 0.9 keeps 37 tokens

"To run a job on the cluster you first have to"



0.17

78%

110 tokens

Temperature rescales the bars before anything is cut

"The GPU ran out of", and the probability it gives to "memory"

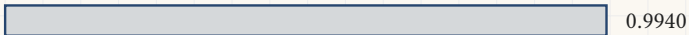
Temperature 0.5



Temperature rescales the bars before anything is cut

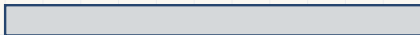
"The GPU ran out of", and the probability it gives to "memory"

Temperature 0.5



0.9940

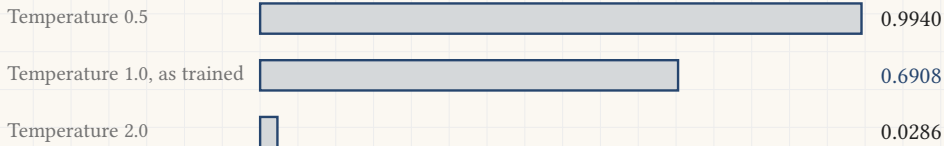
Temperature 1.0, as trained



0.6908

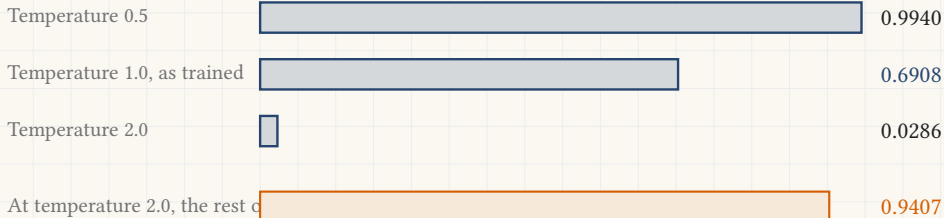
Temperature rescales the bars before anything is cut

"The GPU ran out of", and the probability it gives to "memory"



Temperature rescales the bars before anything is cut

"The GPU ran out of", and the probability it gives to "memory"



Every strategy is a mask over the same vector

logits $\longrightarrow$ probabilities

Every strategy is a mask over the same vector

$$\tilde{p} = m \odot p$$

Every strategy is a mask over the same vector

$$\tilde{p} = \underbrace{m}_{\text{keep or zero}} \odot \underbrace{p}_{151,936 \text{ numbers}}, \text{ then renormalise}$$

Every strategy is a mask over the same vector

$$\tilde{p} = \underbrace{m}_{\text{keep or zero}} \odot \underbrace{p}_{151,936 \text{ numbers}}, \text{ then renormalise}$$

What each mask costs, per token, on the same vector

Take the largest

One pass

Every strategy is a mask over the same vector

$$\tilde{p} = \underbrace{m}_{\text{keep or zero}} \odot \underbrace{p}_{151,936 \text{ numbers}}, \text{ then renormalise}$$

What each mask costs, per token, on the same vector

Take the largest

One pass

Keep the top 50

A partial selection

Every strategy is a mask over the same vector

$$\tilde{p} = \underbrace{m}_{\text{keep or zero}} \odot \underbrace{p}_{151,936 \text{ numbers}}, \text{ then renormalise}$$

What each mask costs, per token, on the same vector

Take the largest

One pass

Keep the top 50

A partial selection

Keep the smallest set reaching 0.9

A full sort of all 151,936

A grammar is the same mask

Compile the schema to an automaton, then zero what cannot come next

A grammar is the same mask

Compile the schema to an automaton, then zero what cannot come next

{ "n ame ":"

A grammar is the same mask

Compile the schema to an automaton, then zero what cannot come next



Only a string may follow,
so every other token is zeroed.

A grammar is the same mask

Compile the schema to an automaton, then zero what cannot come next



Only a string may follow,
so every other token is zeroed.

Mask computed per token

under 40 microseconds

A grammar is the same mask

Compile the schema to an automaton, then zero what cannot come next



Only a string may follow,
so every other token is zeroed.

Mask computed per token

under 40 microseconds

Time per output token, unconstrained

6.2 ms

A grammar is the same mask

Compile the schema to an automaton, then zero what cannot come next



Only a string may follow,
so every other token is zeroed.

Mask computed per token	under 40 microseconds
Time per output token, unconstrained	6.2 ms
Time per output token, constrained	6.3 ms

A grammar is the same mask

Compile the schema to an automaton, then zero what cannot come next



Only a string may follow,
so every other token is zeroed.

Mask computed per token	under 40 microseconds
Time per output token, unconstrained	6.2 ms
Time per output token, constrained	6.3 ms
Function calls that actually parse	62% to 100%

Why does any of this need truncating at all?

One token in a hundred goes wrong. What is the chance that a 1024-token answer has none?

Why does any of this need truncating at all?

One token in a hundred goes wrong. What is the chance that a 1024-token answer has none?

About 1 in 3

About 1 in 30

About 1 in 3000

About 1 in 30000

2

Section 2

Prefill and decode

Two hundred tokens in, twenty tokens out

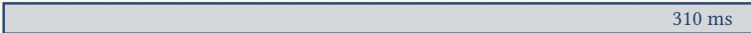
An RTX 4090, published

200 tokens in  10 ms

Two hundred tokens in, twenty tokens out

An RTX 4090, published

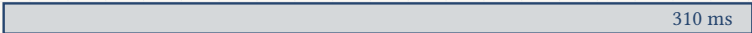
200 tokens in  10 ms

20 tokens out  310 ms

Two hundred tokens in, twenty tokens out

An RTX 4090, published

200 tokens in  10 ms

20 tokens out  310 ms

One GH200 of Santis, measured on 23 September 2026

256 tokens in  7.42 ms

Two hundred tokens in, twenty tokens out

An RTX 4090, published

200 tokens in  10 ms

20 tokens out  310 ms

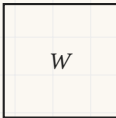
One GH200 of Santis, measured on 23 September 2026

256 tokens in  7.42 ms

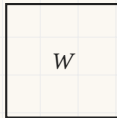
256 tokens out  1111 ms

Prefill is a GEMM, decode is a GEMV

Prefill, one prompt

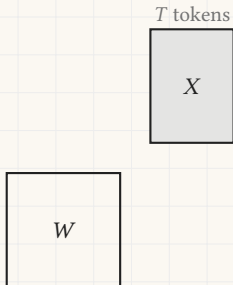


Decode, one token

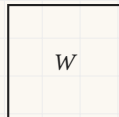


Prefill is a GEMM, decode is a GEMV

Prefill, one prompt

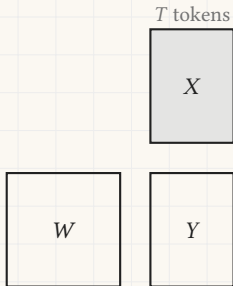


Decode, one token

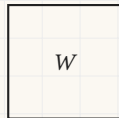


Prefill is a GEMM, decode is a GEMV

Prefill, one prompt

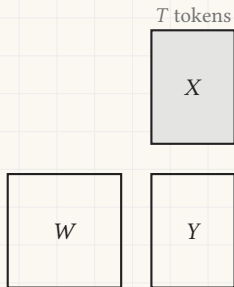


Decode, one token

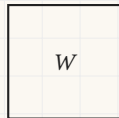


Prefill is a GEMM, decode is a GEMV

Prefill, one prompt

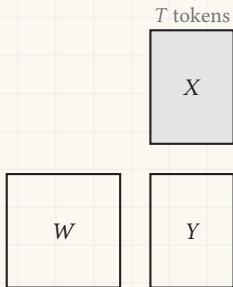


Decode, one token



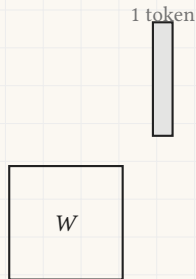
Prefill is a GEMM, decode is a GEMV

Prefill, one prompt



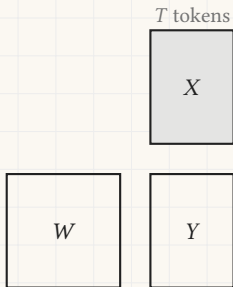
One read of W ,
 T tokens of work

Decode, one token



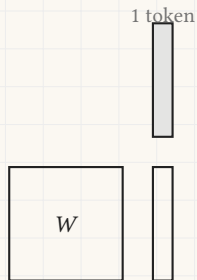
Prefill is a GEMM, decode is a GEMV

Prefill, one prompt



One read of W ,
 T tokens of work

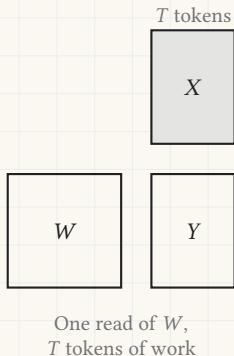
Decode, one token



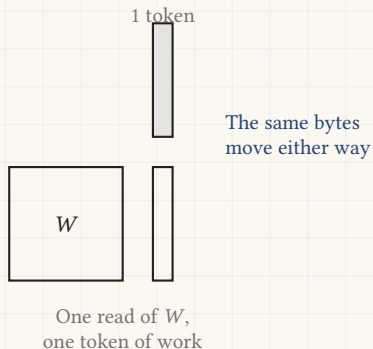
One read of W ,
one token of work

Prefill is a GEMM, decode is a GEMV

Prefill, one prompt



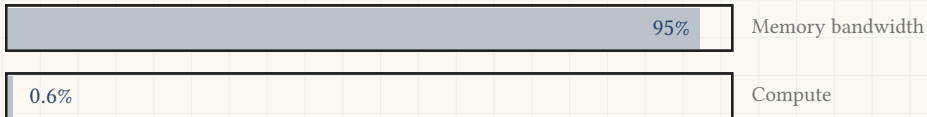
Decode, one token



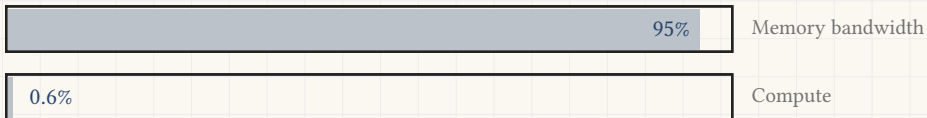
At batch one, one resource is full and the other is idle



At batch one, one resource is full and the other is idle



At batch one, one resource is full and the other is idle



3.22 of 3.37 TB/s

3.2 of 535 TFLOP/s

One GH200 of Santis, 23 September 2026

The floor is the weights over the bandwidth

$$t_{\text{step}} = \frac{B}{W}$$

The floor is the weights over the bandwidth

$$t_{\text{step}} = \frac{\overbrace{\text{bytes of weights}}^B}{W}$$

The floor is the weights over the bandwidth

$$t_{\text{step}} = \frac{\overbrace{\text{bytes of weights}}^B}{\underbrace{\text{bytes per second}}_W}$$

The floor is the weights over the bandwidth

$$t_{\text{step}} = \frac{\underbrace{13.96 \text{ GB}}_{\text{bytes of weights}}}{\underbrace{3.37 \text{ TB/s}}_{\text{bytes per second}}} = 4.14 \text{ ms}$$

The floor is the weights over the bandwidth

$$t_{\text{step}} = \frac{\underbrace{13.96 \text{ GB}}_{\text{bytes of weights}}}{\underbrace{3.37 \text{ TB/s}}_{\text{bytes per second}}} = 4.14 \text{ ms}$$

Measured on Santis: 4.34 ms, which is 95% of the floor.

The floor is the weights over the bandwidth

$$t_{\text{step}} = \frac{\underbrace{13.96 \text{ GB}}_{\text{bytes of weights}}}{\underbrace{3.37 \text{ TB/s}}_{\text{bytes per second}}} = 4.14 \text{ ms}$$

Measured on Santis: 4.34 ms, which is 95% of the floor.

Ridge point of this GH200

159 FLOP per byte

A decode step sits at 1

One step at batch one costs 4.34 ms. What does batch 64 cost?

About 64 times as long

One step at batch one costs 4.34 ms. What does batch 64 cost?

About 64 times as long

About 8 times as long

One step at batch one costs 4.34 ms. What does batch 64 cost?

About 64 times as long

About 8 times as long

About twice as long

One step at batch one costs 4.34 ms. What does batch 64 cost?

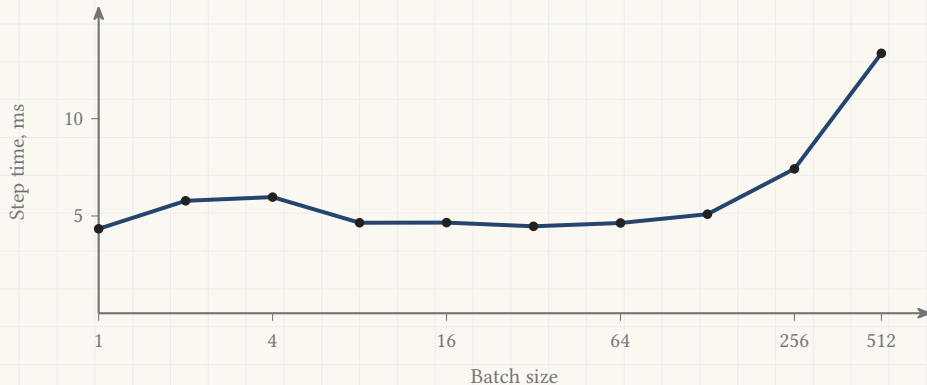
About 64 times as long

About 8 times as long

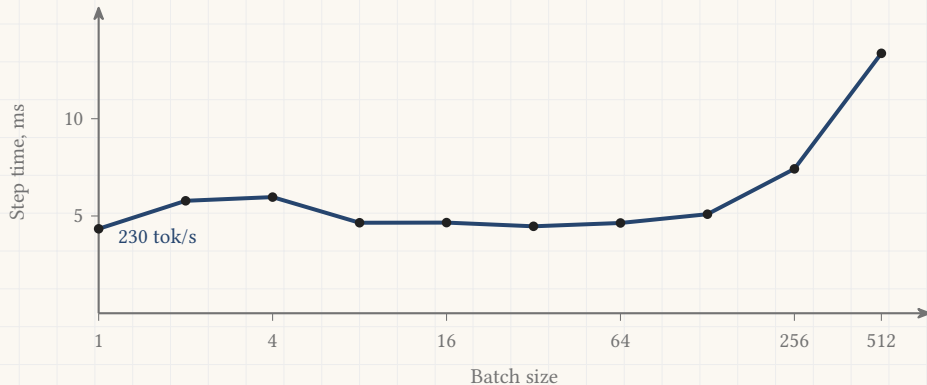
About twice as long

About the same

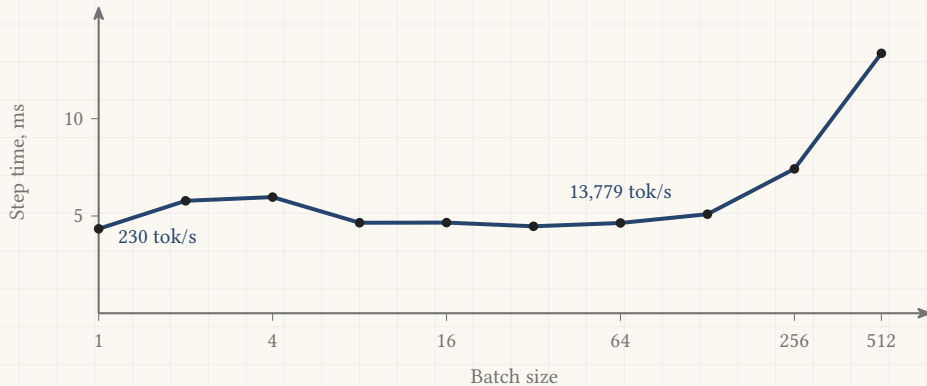
The step time does not move until batch 128



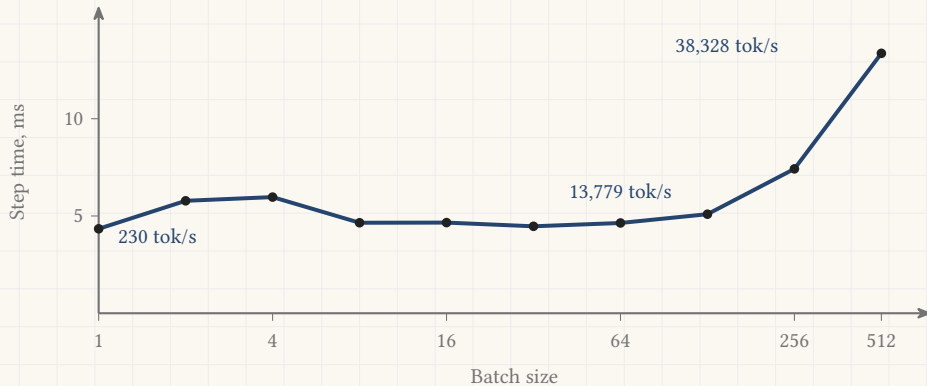
The step time does not move until batch 128



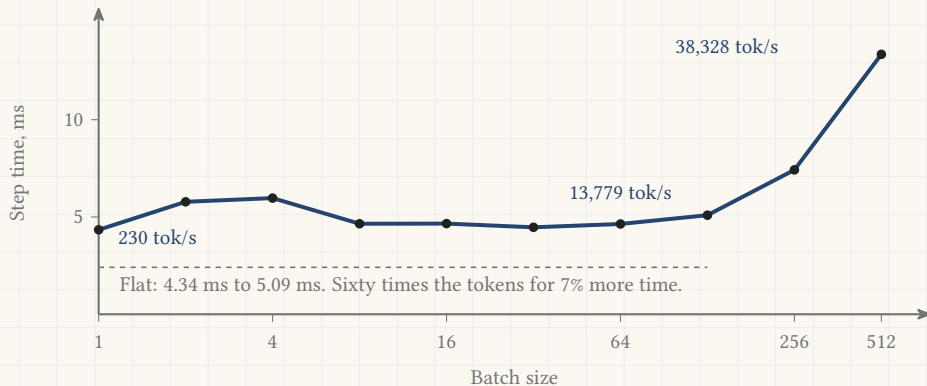
The step time does not move until batch 128



The step time does not move until batch 128



The step time does not move until batch 128

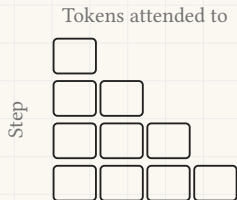


3

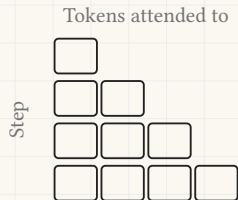
Section 3

The KV cache

Without a cache, every step redoes the whole prompt

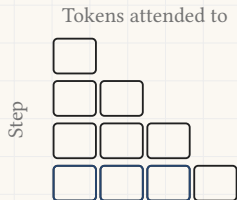


Without a cache, every step redoes the whole prompt



Step 4 recomputes the keys and values of tokens 1 to 3 for the fourth time.

Without a cache, every step redoes the whole prompt



Step 4 recomputes the keys and values of tokens 1 to 3 for the fourth time.

Keep them instead. That store is the KV cache.

The cache is a product of six factors

$$2 \times L \times H_{kv} \times d_h \times b$$

The cache is a product of six factors

$$\underbrace{2}_{K \text{ and } V} \times L \times H_{kv} \times d_h \times b$$

The cache is a product of six factors

$$\underbrace{2}_{K \text{ and } V} \times \underbrace{L}_{\text{layers}} \times H_{kv} \times d_h \times b$$

The cache is a product of six factors

$$\underbrace{2}_{K \text{ and } V} \times \underbrace{L}_{\text{layers}} \times \underbrace{H_{kv}}_{\text{KV heads}} \times d_h \times b$$

The cache is a product of six factors

$$\underbrace{2}_{K \text{ and } V} \times \underbrace{L}_{\text{layers}} \times \underbrace{H_{kv}}_{\text{KV heads}} \times \underbrace{d_h}_{\text{head width}} \times \underbrace{b}_{\text{bytes}}$$

The cache is a product of six factors

$$\underbrace{2}_{K \text{ and } V} \times \underbrace{28}_{\text{layers}} \times \underbrace{4}_{\text{KV heads}} \times \underbrace{128}_{\text{head width}} \times \underbrace{2}_{\text{bytes}} = 57,344 \text{ bytes}$$

The cache is a product of six factors

$$\underbrace{2}_{K \text{ and } V} \times \underbrace{28}_{\text{layers}} \times \underbrace{4}_{\text{KV heads}} \times \underbrace{128}_{\text{head width}} \times \underbrace{2}_{\text{bytes}} = 57,344 \text{ bytes}$$

Qwen2.5-7B-Instruct, from its own configuration file. Per token, per sequence.

The cache is a product of six factors

$$\underbrace{2}_{K \text{ and } V} \times \underbrace{28}_{\text{layers}} \times \underbrace{4}_{\text{KV heads}} \times \underbrace{128}_{\text{head width}} \times \underbrace{2}_{\text{bytes}} = 57,344 \text{ bytes}$$

Qwen2.5-7B-Instruct, from its own configuration file. Per token, per sequence.

56.0 KiB for one token

The cache is a product of six factors

$$\underbrace{2}_{K \text{ and } V} \times \underbrace{28}_{\text{layers}} \times \underbrace{4}_{KV \text{ heads}} \times \underbrace{128}_{\text{head width}} \times \underbrace{2}_{\text{bytes}} = 57,344 \text{ bytes}$$

Qwen2.5-7B-Instruct, from its own configuration file. Per token, per sequence.

56.0 KiB for one token

OPT-13B, for comparison 800 KB per token

Instantiate it until it stops fitting

One sequence of 8192 tokens

0.47 GB

Instantiate it until it stops fitting

One sequence of 8192 tokens

0.47 GB

Sixty-four of them

30.1 GB

Instantiate it until it stops fitting

One sequence of 8192 tokens	0.47 GB
-----------------------------	---------

Sixty-four of them	30.1 GB
--------------------	---------

The same model with 28 KV heads in place of 4, which is what it would cost with no grouped-query attention	210.5 GB
---	----------

Instantiate it until it stops fitting

One sequence of 8192 tokens

0.47 GB

Sixty-four of them

30.1 GB

The same model with 28 KV heads in place of 4,
which is what it would cost with no grouped-query attention

210.5 GB

A GH200 has
95 GiB

Instantiate it until it stops fitting

One sequence of 8192 tokens

0.47 GB

Sixty-four of them

30.1 GB

The same model with 28 KV heads in place of 4,
which is what it would cost with no grouped-query attention

210.5 GB

A GH200 has
95 GiB

Measured, per sample

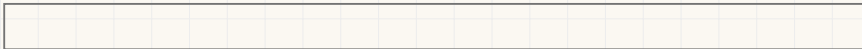
multi-head 1.51 s

grouped-query 0.28 s

quality 47.2 to 47.1

What the cache does to the budget

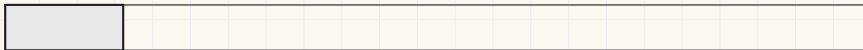
One GH200, 95 GiB



What the cache does to the budget

One GH200, 95 GiB

13.96 GB



Weights

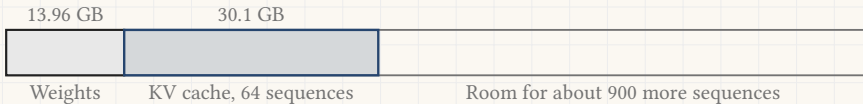
What the cache does to the budget

One GH200, 95 GiB



What the cache does to the budget

One GH200, 95 GiB



What the cache does to the budget

One GH200, 95 GiB



What is left over, over the per-token cost

88 GB over 56.0 KiB is 1.54 million tokens

Reserving for the worst case wastes most of it



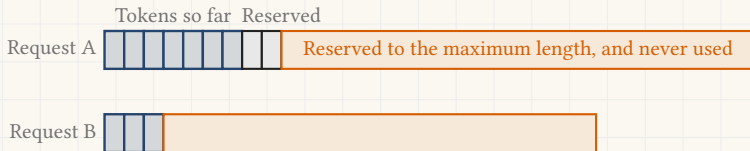
Reserving for the worst case wastes most of it



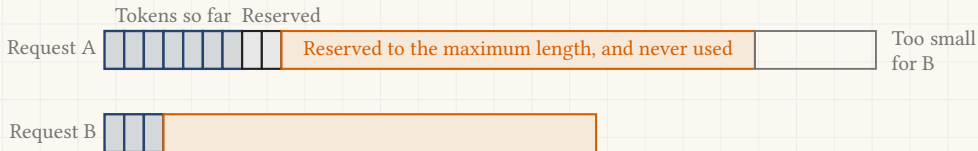
Reserving for the worst case wastes most of it



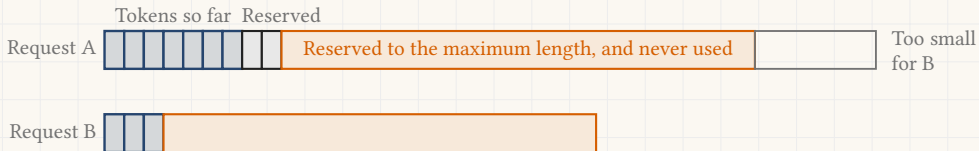
Reserving for the worst case wastes most of it



Reserving for the worst case wastes most of it



Reserving for the worst case wastes most of it



20.4% to 38.2% of the KV cache holds actual token states, in the systems vLLM measured

Pages, and a block table

Logical blocks

Alan Turing is a

computer scientist

Pages, and a block table

Logical blocks

Alan Turing is a

computer scientist

Block table

$0 \rightarrow 7$

$1 \rightarrow 1$

Pages, and a block table

Logical blocks

Alan Turing is a

computer scientist

Block table

0 $\rightarrow$ 7

1 $\rightarrow$ 1

Physical blocks



Pages, and a block table

Logical blocks

Alan Turing is a

computer scientist

Block table

0 $\rightarrow$ 7

1 $\rightarrow$ 1

Physical blocks

7

1

Pages, and a block table

Logical blocks

Alan Turing is a

computer scientist

Block table

0 $\rightarrow$ 7

1 $\rightarrow$ 1

Physical blocks

7

1

A block is a page, so the waste per request is bounded by one block

Pages, and a block table

Logical blocks

Alan Turing is a

computer scientist

Block table

0 $\rightarrow$ 7

1 $\rightarrow$ 1

Physical blocks

7

1

A block is a page, so the waste per request is bounded by one block

96.3% utilisation, and 2.2 times the concurrent requests

Pages, and a block table

Logical blocks

Alan Turing is a

computer scientist

Block table

0 $\rightarrow$ 7

1 $\rightarrow$ 1

Physical blocks

7

1

A block is a page, so the waste per request is bounded by one block

96.3% utilisation, and 2.2 times the concurrent requests

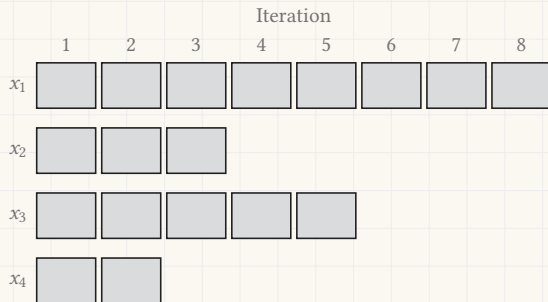
The bill the attention kernel runs 20% to 26% slower

4

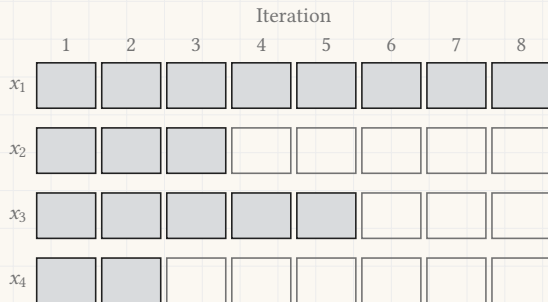
Section 4

Continuous batching

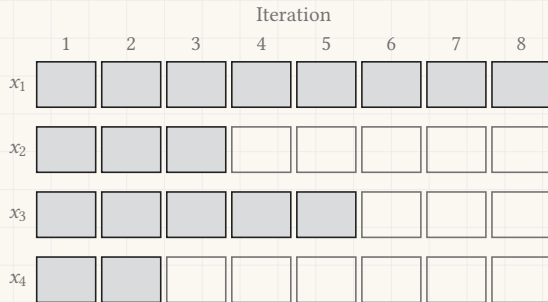
A static batch waits for its slowest member



A static batch waits for its slowest member

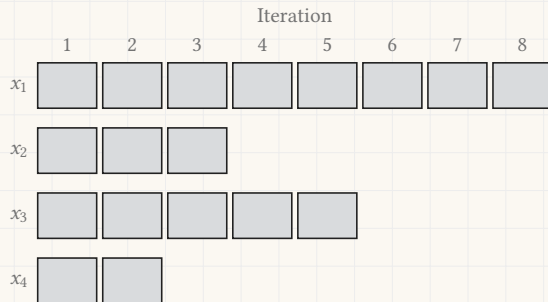


A static batch waits for its slowest member



These cells compute nothing,
and the slot cannot be reused.

A finished row is refilled at the next iteration



A finished row is refilled at the next iteration

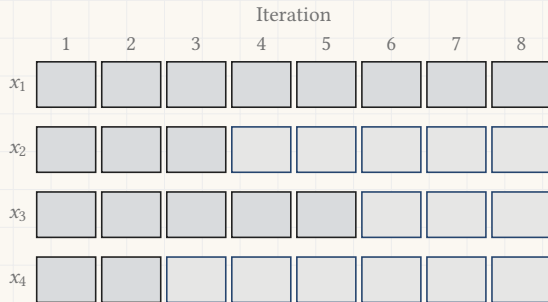
	Iteration							
	1	2	3	4	5	6	7	8
x_1								
x_2								
x_3								
x_4								

A finished row is refilled at the next iteration

	Iteration							
	1	2	3	4	5	6	7	8
x_1								
x_2								
x_3								
x_4								

A new request takes the slot at the next iteration.

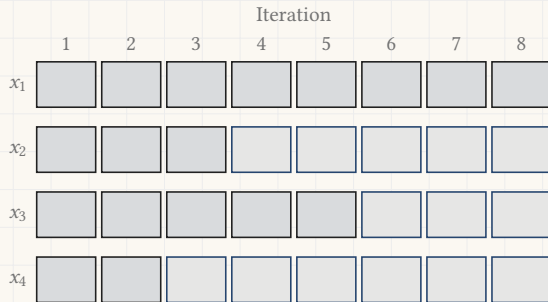
A finished row is refilled at the next iteration



A new request takes the slot
at the next iteration.

Scheduling per iteration, rather than per batch

A finished row is refilled at the next iteration



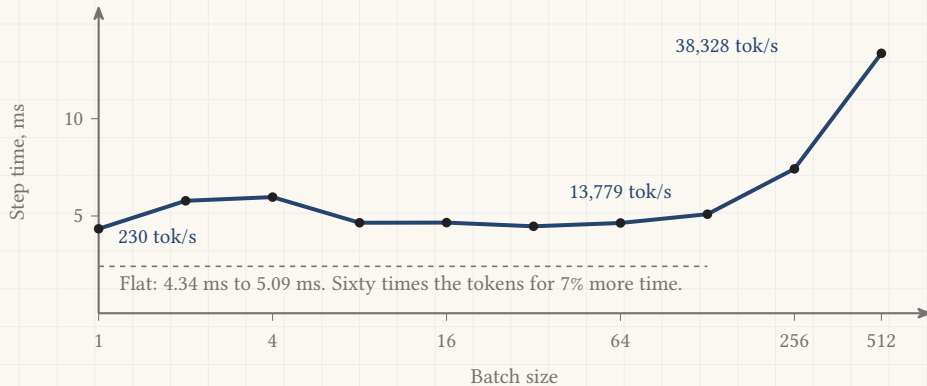
A new request takes the slot at the next iteration.

Scheduling per iteration, rather than per batch

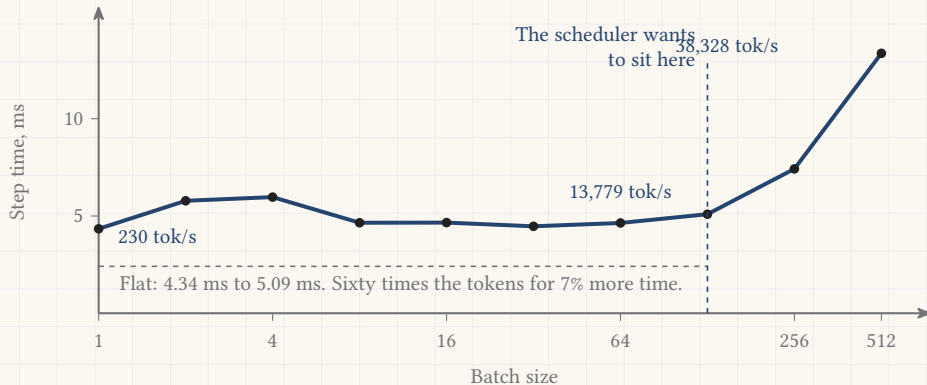
0.185 to 6.81 requests per second

at a matched median latency of 190 ms per token

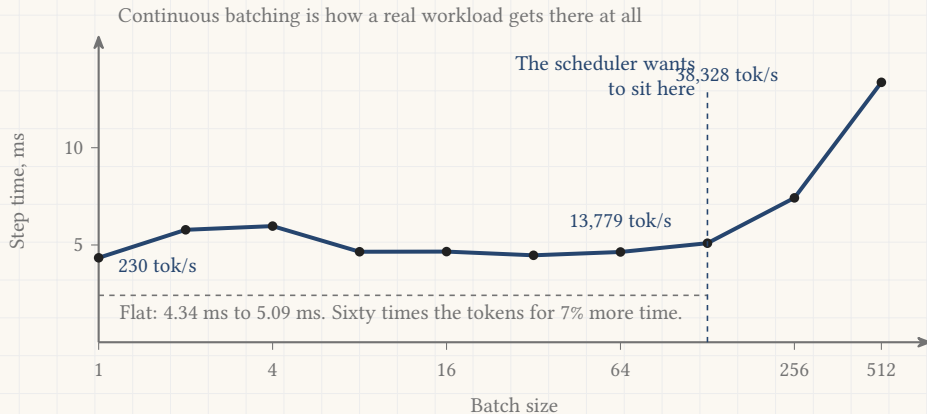
Where the batch size comes from



Where the batch size comes from



Where the batch size comes from



5

Section 5

Quantisation and speculation

Quantisation stores small integers and one scale

bf16

1.456

2.376

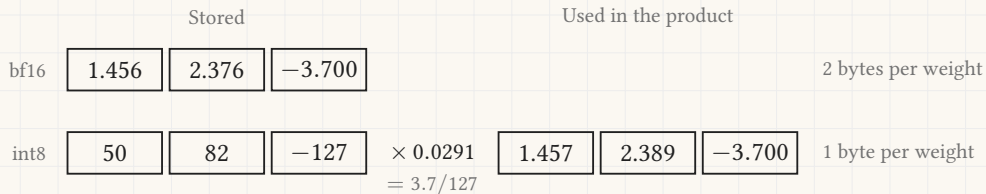
-3.700

2 bytes per weight

Quantisation stores small integers and one scale

	Stored			
bf16	1.456	2.376	-3.700	2 bytes per weight
int8	50	82	-127	$\times 0.0291$ $= 3.7/127$

Quantisation stores small integers and one scale



Quantisation stores small integers and one scale

	Stored				Used in the product			
bf16	1.456	2.376	−3.700					2 bytes per weight
int8	50	82	−127	$\times 0.0291$ $= 3.7/127$	1.457	2.389	−3.700	1 byte per weight
int4	3	4	−7	$\times 0.5286$ $= 3.7/7$	1.586	2.114	−3.700	Half a byte per weight

Quantisation stores small integers and one scale

	Stored				Used in the product			
bf16	1.456	2.376	−3.700					2 bytes per weight
int8	50	82	−127	$\times 0.0291$ $= 3.7/127$	1.457	2.389	−3.700	1 byte per weight
int4	3	4	−7	$\times 0.5286$ $= 3.7/7$	1.586	2.114	−3.700	Half a byte per weight
					Off by up to 0.262			

Half the bytes, half the step

The same 7B weight stack, at batch one, on one GH200 of Santis

fp32

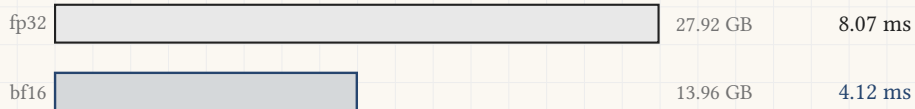


27.92 GB

8.07 ms

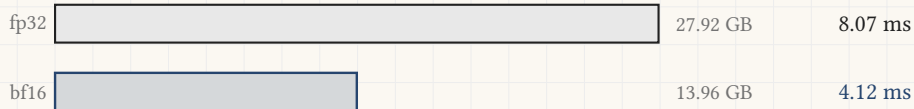
Half the bytes, half the step

The same 7B weight stack, at batch one, on one GH200 of Santis



Half the bytes, half the step

The same 7B weight stack, at batch one, on one GH200 of Santis



Both sustained about 3.4 TB/s

Half the bytes, half the time

The kernel still has to cash the cheque

The same stack again, at batch 32

bf16

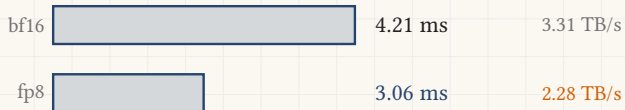


4.21 ms

3.31 TB/s

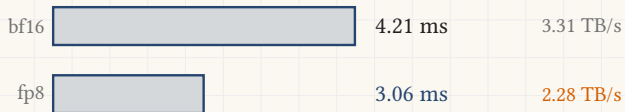
The kernel still has to cash the cheque

The same stack again, at batch 32



The kernel still has to cash the cheque

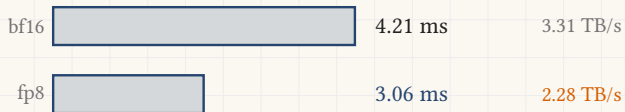
The same stack again, at batch 32



Half the bytes bought a factor of 1.4 where the bytes alone predict 2

The kernel still has to cash the cheque

The same stack again, at batch 32

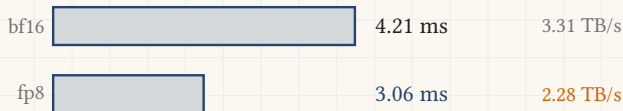


Half the bytes bought a factor of 1.4 where the bytes alone predict 2

Eight-bit matrix multiplication, against fp16

The kernel still has to cash the cheque

The same stack again, at batch 32



Half the bytes bought a factor of 1.4 where the bytes alone predict 2

Eight-bit matrix multiplication, against fp16

model dimension 768	0.14 times	dimension 2048	0.51 times	12288	1.81
---------------------	------------	----------------	------------	-------	------

And the accuracy, on the same frame

WikiText2 perplexity, OPT-175B

16-bit	8.34
--------	------

And the accuracy, on the same frame

WikiText2 perplexity, OPT-175B

16-bit	8.34
4-bit, chosen carefully	8.37

And the accuracy, on the same frame

WikiText2 perplexity, OPT-175B

16-bit	8.34
4-bit, chosen carefully	8.37
4-bit, rounded to nearest	10.54

And the accuracy, on the same frame

WikiText2 perplexity, OPT-175B

16-bit	8.34
4-bit, chosen carefully	8.37
4-bit, rounded to nearest	10.54
3-bit, rounded to nearest	7300

And the accuracy, on the same frame

WikiText2 perplexity, OPT-175B

16-bit	8.34
4-bit, chosen carefully	8.37
4-bit, rounded to nearest	10.54
3-bit, rounded to nearest	7300

The method matters more than the bit width

And the accuracy, on the same frame

WikiText2 perplexity, OPT-175B

16-bit	8.34
4-bit, chosen carefully	8.37
4-bit, rounded to nearest	10.54
3-bit, rounded to nearest	7300

The method matters more than the bit width

230 ms to 71 ms per token, and five GPUs to one

Guess several tokens, then check them all at once

Draft model

4B parameters, 1.8 ms per token

Guess several tokens, then check them all at once

Draft model

4B parameters, 1.8 ms per token

the

cat

sat

down

Four guesses, cheaply

Guess several tokens, then check them all at once

Draft model

4B parameters, 1.8 ms per token

the

cat

sat

down

Four guesses, cheaply

Target model

70B parameters, 14.1 ms per token, one forward pass for all four

Guess several tokens, then check them all at once

Draft model

4B parameters, 1.8 ms per token

the

cat

sat

down

Four guesses, cheaply

Target model

70B parameters, 14.1 ms per token, one forward pass for all four

the

cat

sat

down

Three guesses accepted

Guess several tokens, then check them all at once

Draft model

4B parameters, 1.8 ms per token

the cat sat down

Four guesses, cheaply

Target model

70B parameters, 14.1 ms per token, one forward pass for all four

the cat sat down → on

Three guesses accepted the fourth replaced by the target's own token

How many to guess

$$\mathbb{E}[\text{tokens}] = 1$$

How many to guess

$$\mathbb{E}[\text{tokens}] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

How many to guess

$$\mathbb{E}[\text{tokens}] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

How many to guess

$$\mathbb{E}[\text{tokens}] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

α , how often a guess survives

γ , how many the draft proposes

How many to guess

$$\mathbb{E}[\text{tokens}] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

α , how often a guess survives

γ , how many the draft proposes

Measured acceptance rates, a 137B model drafted by

an 8B model

0.75

a 100M model

0.61

a bigram table

0.20

How many to guess

$$\mathbb{E}[\text{tokens}] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

α , how often a guess survives

γ , how many the draft proposes

Measured acceptance rates, a 137B model drafted by

an 8B model

0.75

a 100M model

0.61

a bigram table

0.20

More guesses lower the acceptance rate and lengthen the round

How many to guess

$$\mathbb{E}[\text{tokens}] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

α , how often a guess survives

γ , how many the draft proposes

Measured acceptance rates, a 137B model drafted by

an 8B model	0.75	a 100M model	0.61	a bigram table	0.20
-------------	------	--------------	------	----------------	------

More guesses lower the acceptance rate and lengthen the round

so the total time has a minimum, at three guesses

It beats the bandwidth bound, and the output is identical

Chinchilla 70B, batch one, sixteen TPU v4s, four guesses

Plain decoding 14.1 ms per token

It beats the bandwidth bound, and the output is identical

Chinchilla 70B, batch one, sixteen TPU v4s, four guesses

Plain decoding	14.1 ms per token	
With guessing	5.73 ms per token	2.46 times

It beats the bandwidth bound, and the output is identical

Chinchilla 70B, batch one, sixteen TPU v4s, four guesses

Plain decoding 14.1 ms per token

With guessing 5.73 ms per token 2.46 times

Benchmark score 45.1% to 47.0%

It beats the bandwidth bound, and the output is identical

Chinchilla 70B, batch one, sixteen TPU v4s, four guesses

Plain decoding	14.1 ms per token	
With guessing	5.73 ms per token	2.46 times
Benchmark score	45.1% to 47.0%	

Faster than the memory bandwidth limit for ordinary decoding

It beats the bandwidth bound, and the output is identical

Chinchilla 70B, batch one, sixteen TPU v4s, four guesses

Plain decoding	14.1 ms per token	
With guessing	5.73 ms per token	2.46 times
Benchmark score	45.1% to 47.0%	

Faster than the memory bandwidth limit for ordinary decoding

The output distribution is the target model's own

Which of the three moved the wall?

Continuous batching

Quantisation

Guessing tokens

Which of the three moved the wall?

Continuous batching

Quantisation

Guessing tokens

Which lets you serve a longer context, and which only makes each step cheaper?

6

Section 6

Serving on a cluster

One command starts a server

o o o

```
$ vllm serve Qwen/Qwen2.5-7B-Instruct --host 0.0.0.0 --port  
8000
```

```
INFO Started server process
```

```
INFO Application startup complete.
```

One command starts a server

o o o

```
$ vllm serve Qwen/Qwen2.5-7B-Instruct --host 0.0.0.0 --port  
8000
```

```
INFO Started server process
```

```
INFO Application startup complete.
```

The last line is the readiness signal every centre waits for

One command starts a server

o o o

```
$ vllm serve Qwen/Qwen2.5-7B-Instruct --host 0.0.0.0 --port 8000
```

```
INFO Started server process
```

```
INFO Application startup complete.
```

~~The last line is the readiness signal every centre waits for~~

It speaks the OpenAI protocol under /v1, so every client already works.

One command starts a server

o o o

```
$ vllm serve Qwen/Qwen2.5-7B-Instruct --host 0.0.0.0 --port  
8000
```

```
INFO Started server process
```

```
INFO Application startup complete.
```

~~The last line is the readiness signal every centre waits for~~

It speaks the OpenAI protocol under /v1, so every client already works.

The flags that matter: `--tensor-parallel-size`, `--max-model-len`,
`--gpu-memory-utilization`, `--quantization`, `--kv-cache-dtype`.

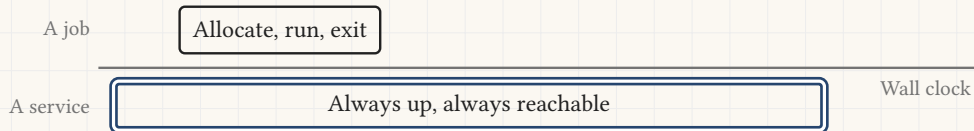
A batch job ends. A service does not.

A job

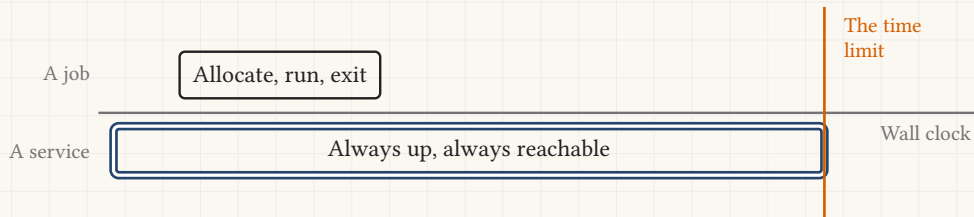
Allocate, run, exit

Wall clock

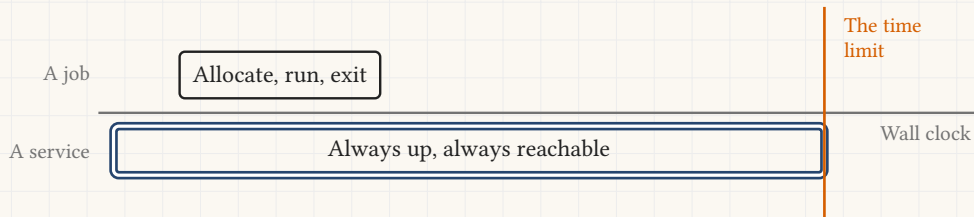
A batch job ends. A service does not.



A batch job ends. A service does not.

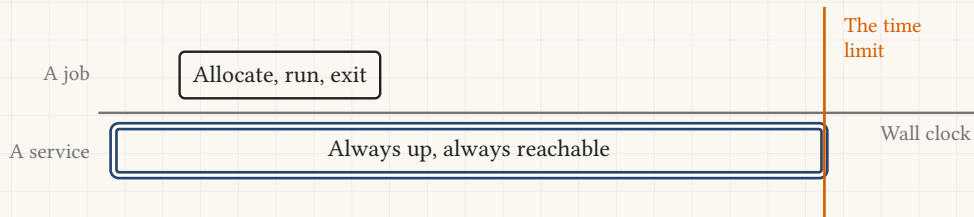


A batch job ends. A service does not.



No centre in the survey solves this inside a job

A batch job ends. A service does not.



No centre in the survey solves this inside a job

The facility answer is to stop using a job

Live, Starting, Queued, Offline

Bind, discover, reach

- 1 Bind, inside the job, on every interface

o o o

```
$ vllm serve $MODEL --host 0.0.0.0 --port 8000
```

Bind, discover, reach

- 1 Bind, inside the job, on every interface

```
ooo
```

```
$ vllm serve $MODEL --host 0.0.0.0 --port 8000
```

- 2 Discover, from Slurm itself

```
ooo
```

```
$ scontrol show hostnames $SLURM_NODELIST | head -n1
```

Bind, discover, reach

- ① Bind, inside the job, on every interface

```
ooo
```

```
$ vllm serve $MODEL --host 0.0.0.0 --port 8000
```

- ② Discover, from Slurm itself

```
ooo
```

```
$ scontrol show hostnames $SLURM_NODELIST | head -n1
```

- ③ Reach, from a laptop, through the login node

```
ooo
```

```
$ ssh login.example.org -NL 8000:$NODE:8000
```

Four to a node

`--gpu-memory-utilization` defaults to 0.92, and its own documentation says:

Four to a node

`--gpu-memory-utilization` defaults to 0.92, and its own documentation says:

"It does not matter if you have another vLLM instance running on the same GPU."

Four to a node

`--gpu-memory-utilization` defaults to 0.92, and its own documentation says:

"It does not matter if you have another vLLM instance running on the same GPU."

Two centres, two answers to sharing a node:

Four to a node

`--gpu-memory-utilization` defaults to 0.92, and its own documentation says:

"It does not matter if you have another vLLM instance running on the same GPU."

Two centres, two answers to sharing a node:

Refuse the port. Bind a Unix socket inside the job's own scratch, so no two jobs can collide and no key is needed.

Four to a node

`--gpu-memory-utilization` defaults to 0.92, and its own documentation says:

"It does not matter if you have another vLLM instance running on the same GPU."

Two centres, two answers to sharing a node:

Refuse the port. Bind a Unix socket inside the job's own scratch, so no two jobs can collide and no key is needed.

Keep the port and mint an API key for each deployment.

What to watch while it runs

The server exports Prometheus metrics at `/metrics`

ooo

```
vllm:kv_cache_usage_perc  
vllm:num_requests_running  
vllm:request_queue_time_seconds  
vllm:time_to_first_token_seconds
```

What to watch while it runs

The server exports Prometheus metrics at /metrics

ooo

```
vllm:kv_cache_usage_perc  
vllm:num_requests_running  
vllm:request_queue_time_seconds
```

The first one tells you whether to raise the batch size

What to watch while it runs

The server exports Prometheus metrics at /metrics

ooo

```
vllm:kv_cache_usage_perc  
vllm:num_requests_running  
vllm:request_queue_time_seconds  
vllm:time_to_first_token_seconds  
vllm:time_per_output_token_seconds
```

The benchmark reports, at the mean, median and 99th percentile

time to first token

time per output token

the gaps between them

What to watch while it runs

The server exports Prometheus metrics at /metrics

ooo

```
vllm:kv_cache_usage_perc  
vllm:num_requests_running  
vllm:request_queue_time_seconds  
vllm:time_to_first_token_seconds  
vllm:time_per_output_token_seconds
```

The benchmark reports, at the mean, median and 99th percentile

time to first token

time per output token

the gaps between them

Run it twice and the prefix cache inflates the second result